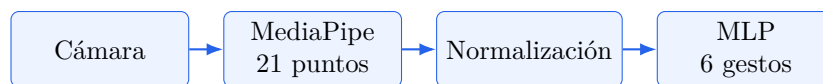


Reconocimiento de gestos de mano

Del dataset grabado por el grupo al modelo que se ejecuta en el navegador



Javier Saguar · Alejandro Cuevas · Mónica Fernández
Pedro José Orrego · Daniel Naval

Resumen

Esta memoria describe la primera fase del proyecto: un reconocedor de seis gestos de mano a partir de la cámara de un portátil. Grabamos un dataset propio de 3000 imágenes con cinco personas, extrajimos los 21 puntos de la mano con MediaPipe y comparamos siete familias de modelos con cuatro normalizaciones geométricas. Para medir de forma honesta evaluamos siempre con una persona que el modelo no ha visto (*leave-one-participant-out*, LOPO). La normalización respecto a la muñeca, la escala y el giro es la decisión que más influye: sube la CNN del enunciado del 90,9 % al 95,8 %. El modelo elegido es un perceptrón multicapa (MLP) de 5030 parámetros que alcanza un 96,5 % en LOPO, ocupa 86 KB y clasifica en 0,84 ms; se exporta a JSON y se ejecuta en el navegador con las mismas probabilidades que en Python. Un análisis sin etiquetas confirma que la normalización ya separa los gestos, aunque cada uno está formado por varias variantes que solo un clasificador supervisado une.

Índice

1. Objetivo	3
2. Grabación del dataset	3
2.1. Los seis gestos	3
2.2. Kit de grabación	3
2.3. Reglas de calidad	4
2.4. Resultado de la grabación	4
2.5. Privacidad y copia de seguridad	5
3. De la imagen a la representación	5
3.1. Puntos de la mano en vez de píxeles	5
3.2. Corrección del aspecto	5
3.3. Normalizaciones	5
3.4. Invarianza a la mano y aumentación	6
4. Modelos	6
5. Cómo evaluamos	6
5.1. Protocolos	6
5.2. Métricas	7
6. Resultados	7
6.1. La normalización es la decisión que más pesa	7
6.2. Evaluar con personas nuevas	8
6.3. Experimentos de invarianza y aumentación	8
6.4. Errores por gesto	9
6.5. Acierto por persona	9
6.6. Coste computacional	10
7. Modelo elegido y despliegue	11

8. Análisis sin etiquetas	11
9. Limitaciones	13
10. Conclusiones	13
A. Reproducibilidad	14

1 Objetivo

El enunciado pide reconocer gestos de mano con MediaPipe y un clasificador entrenado sobre un dataset grabado por el grupo, y usarlos para dar órdenes. Nos fijamos tres objetivos:

1. **Un dataset propio y variado**, con varias personas, distancias, ángulos y posiciones, grabado de forma que se pueda evaluar con personas nuevas.
2. **Una evaluación honesta**: medir el acierto con alguien a quien el modelo no ha visto nunca, que es lo que ocurre en una demostración.
3. **Un modelo ligero**, capaz de funcionar en tiempo real en el portátil y también en el navegador, para integrarlo después con la plataforma de datos y los chatbots (fases 2 y 3) sin enviar ninguna imagen.

2 Grabación del dataset

2.1 Los seis gestos

Elegimos seis gestos fáciles de hacer y de explicar. Cada uno tiene asignada una orden del vehículo tanque del enunciado (en el portal web de la fase 2 tienen otras acciones, como cambiar de sección o abrir el asistente).

Clase	Gesto	Cómo se hace	Orden del tanque
ok	OK	Pulgar e índice en círculo; los otros tres dedos extendidos y separados	GIRAR
paper	Papel	Mano abierta, dedos extendidos y separados, palma hacia la cámara	PARAR
rock	Piedra	Puño cerrado con el pulgar recogido por delante	RETROCEDER
rockandroll	Cuernos	Índice y meñique extendidos; corazón y anular doblados	DISPARAR
scissors	Tijera	Índice y corazón en V bien abierta; el resto cerrado	HACER FOTO
thumbsup	Pulgar arriba	Puño con el pulgar extendido hacia arriba	AVANZAR

Tabla 1: Gestos del dataset.

Dos pares se parecen mucho y la guía de grabación los destacaba: **rock** y **thumbsup**, que solo se diferencian en el pulgar, y **scissors** y **rockandroll**. Como se verá en la sección 6, son precisamente los que más se confunden.

2.2 Kit de grabación

Partimos del grabador del repositorio de la asignatura ([efhes/IE-workspace](#)) y lo adaptamos para que cada persona grabara en Windows con la webcam de su portátil, sin Raspberry Pi. El kit ([kit_grabacion/](#)) se instala con Python 3.12 y funciona así:

- Al arrancar pide un **identificador de participante** (p1 a p5) y la mano con la que se va a grabar. El identificador es la base de la evaluación por persona, así que cada uno usa siempre el suyo.
- Para cada gesto, la ventana muestra los puntos detectados por MediaPipe; solo cuando se ven se pulsa **s** y empieza la captura: **una imagen por segundo durante 100 segundos**.

- Cada toma se guarda en una carpeta nueva con fecha y hora y un `metadata.json` con su estado (completa, interrumpida o en curso), de modo que ninguna grabación se pisa.
- Un modo de prueba graba 5 imágenes por gesto en otra carpeta para comprobar cámara, luz y detección.

2.3 Reglas de calidad

Si la persona se queda quieta, las 100 imágenes de un gesto son casi el mismo fotograma y el modelo memoriza una postura. Por eso la guía pedía moverse despacio y de forma continua durante los 100 segundos: acercarse a unos 30 cm y alejarse hasta 1 m, girar e inclinar la muñeca y recorrer el encuadre, siempre con la misma mano.

2.4 Resultado de la grabación

Cinco personas grabaron una toma completa cada una: 6 gestos $\times$ 100 imágenes $\times$ 5 personas = 3000 **imágenes**. Otras cuatro tomas quedaron interrumpidas o vacías (0, 6, 0 y 0 imágenes) y se descartan automáticamente por su estado. MediaPipe detecta la mano en 2931 **imágenes** (97,7 %); las imágenes sin mano no se usan.

Participante	Imágenes	Con mano	Detección	Tamaño de la mano (p5–p95)	Máx./mín.
p1	600	572	95,3 %	0,10 – 0,26	2,5 $\times$
p2	600	585	97,5 %	0,14 – 0,41	2,9 $\times$
p3	600	588	98,0 %	0,17 – 0,29	1,7 $\times$
p4	600	599	99,8 %	0,16 – 0,30	1,9 $\times$
p5	600	587	97,8 %	0,11 – 0,24	2,1 $\times$

Tabla 2: Tomas válidas. El tamaño de la mano es la distancia de la muñeca al nudillo del corazón, en unidades de alto de imagen: cuanto mayor, más cerca de la cámara.

La figura 1 confirma que la grabación es variada: el tamaño de la mano cambia entre 1,7 y 2,9 veces dentro de cada toma y la inclinación respecto a la vertical recorre unos $\pm 70^\circ$. La detección más baja es la de *rock* (89 % en p5 y 92 % en p1): con el puño cerrado hay menos dedos visibles.

Diversidad de la grabación por participante

Cajas: del percentil 25 al 75; línea: mediana; bigotes: 1,5 veces el rango intercuartílico. Rangos amplios = distintas distancias y ángulos.

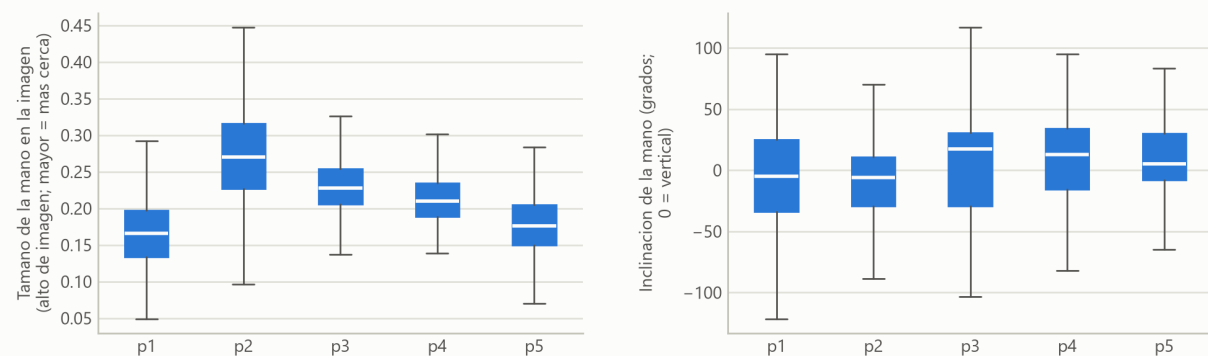


Figura 1: Diversidad de distancias (izquierda) y de inclinaciones (derecha) por participante.

La etiqueta de mano izquierda o derecha que devuelve MediaPipe no es fiable en este dataset: ronda el 50 % en todas las tomas aunque cada persona grabó con una sola mano, porque al girar la muñeca el dorso de una mano tiene en 2D la misma forma que la palma de la otra.

2.5 Privacidad y copia de seguridad

Las imágenes muestran la cara de los cinco integrantes, así que **no se publican** en el repositorio, que es público. Se guardan en una copia comprimida y verificada (`dataset_gestos_PIDS_2026-09-16.zip`, 599 MB, con el SHA-256 de cada fichero) que se comparte solo con el grupo y los profesores. Por el mismo motivo, la figura de errores con fotos que genera el pipeline tampoco se publica. En el repositorio están los puntos de la mano extraídos (`datos_generados/`), que es todo lo que necesitan los modelos.

3 De la imagen a la representación

3.1 Puntos de la mano en vez de píxeles

MediaPipe Hands localiza 21 puntos de la mano (muñeca, nudillos y articulaciones de cada dedo). Clasificamos esas coordenadas y no la imagen por tres motivos:

- **Generaliza mejor:** los puntos no dependen del fondo, la luz, la ropa ni el tono de piel, de modo que el modelo aprende la forma de la mano y no el entorno de quien grabó.
- **Es pequeño:** 42 números por mano (x, y de 21 puntos) frente a cientos de miles de píxeles.
- **No cuesta tiempo:** MediaPipe tarda 16,1 ms por fotograma de 1280×720 , y eso ya fija el ritmo; el clasificador añade menos de 1 ms (sección 6.6).

3.2 Corrección del aspecto

MediaPipe devuelve x dividida por el ancho de la imagen e y dividida por el alto. En una imagen de 1280×720 , una unidad de x mide 1,78 veces lo que una de y , así que medir distancias o girar sobre esas coordenadas deforma la mano. Antes de nada se pasa a un espacio con el aspecto corregido:

$$\tilde{x}_i = x_i \cdot \frac{\text{ancho}}{\text{alto}}, \quad \tilde{y}_i = y_i, \quad i = 0, \dots, 20.$$

3.3 Normalizaciones

Probamos cuatro representaciones, cada una añadiendo un paso a la anterior. Sea $\mathbf{p}_i$ el punto i con el aspecto corregido, $\mathbf{p}_0$ la muñeca y $\mathbf{p}_9$ el nudillo del dedo corazón:

1. **Ninguna:** las coordenadas de MediaPipe tal cual, como hace el notebook de la asignatura. La posición de la mano en el encuadre pasa a ser un dato más, aunque no debería influir en el gesto.
2. **Muñeca:** $\mathbf{q}_i = \mathbf{p}_i - \mathbf{p}_0$. Quita la posición en el encuadre.
3. **Muñeca + escala:** $\mathbf{q}_i = (\mathbf{p}_i - \mathbf{p}_0)/s$, con $s = \|\mathbf{p}_9 - \mathbf{p}_0\|$. Esa distancia apenas cambia entre gestos porque la palma no se deforma, así que quita la dependencia de la distancia a la cámara.
4. **Muñeca + escala + giro:** además se gira la mano para que el vector muñeca–nudillo del corazón apunte siempre hacia arriba. Quita la inclinación. A cambio, dos gestos que solo se distinguieran por su orientación (pulgar arriba y pulgar abajo) dejarían de poder separarse; entre nuestros seis gestos no ocurre.

3.4 Invarianza a la mano y aumentación

Cada persona grabó con una mano, pero en la demo se puede usar cualquiera. Probamos dos estrategias alternativas:

- **Espejo:** reflejar las manos que MediaPipe etiqueta como izquierdas para llevarlas todas a la misma lateralidad. Depende de una etiqueta que, como vimos, no es fiable.
- **Aumentación con volteo:** añadir al entrenamiento copias perturbadas de cada mano (giro aleatorio de hasta $\pm 15^\circ$, escala de $\pm 10\%$, desplazamiento de hasta 0,05 y ruido gaussiano en cada punto, que imita el temblor de MediaPipe) y voltear horizontalmente la mitad. El modelo aprende las dos manos solo con datos.

La misma cadena de transformaciones se aplica al entrenar, en la demo de Windows y en el navegador.

4 Modelos

Comparamos siete familias. Cada una responde a una pregunta distinta sobre el problema:

Modelo	Arquitectura y motivo
CNN del enunciado	Entrada 21×2 ; dos capas Conv2D de 16 filtros con núcleo que recorre 5 puntos, ReLU, Dense(32) y softmax. Es la referencia que pide el enunciado.
CNN del notebook	La que construye de verdad el notebook de la asignatura: una sola Conv2D y <i>dropout</i> de 0,3. Se entrena en el propio notebook, como comprobación.
MLP	Perceptrón multicapa $42 \rightarrow 64 \rightarrow 32 \rightarrow 6$ con <i>dropout</i> de 0,2. Pregunta: si la entrada ya es geometría, ¿hacen falta convoluciones?
SVM	Núcleo RBF ($C = 10$) sobre las coordenadas estandarizadas. Fuerte con pocos datos y pocas dimensiones.
Random forest	200 árboles. No lineal y sin necesidad de escalar; grande en disco.
Regresión logística	Base lineal: ¿basta una frontera recta tras normalizar?
k vecinos	$k = 5$, sin entrenamiento: clasifica por las 5 manos más parecidas del entrenamiento. Cota inferior sencilla.

Tabla 3: Modelos comparados.

Las redes usan los hiperparámetros del notebook (AdamW con tasa 0,001, *weight decay* de 10^{-4} , lotes de 32 y parada temprana sobre la pérdida de validación) con una diferencia: hasta 200 épocas con paciencia 20 en lugar de 300 y 50, porque la mejor época llegaba de media en la 23 en LOPO. Todo se entrenó con Keras 3 sobre PyTorch en una GPU NVIDIA RTX 5070 de portátil, y scikit-learn 1.9 para los modelos clásicos.

5 Cómo evaluamos

5.1 Protocolos

- **Aleatorio:** 70/30 al azar sobre todos los fotogramas. Imágenes casi idénticas, grabadas con un segundo de diferencia, caen a ambos lados, así que da una cifra optimista. Está implementado para enseñar ese sesgo, pero no lo usamos para decidir.
- **Temporal:** la partición del grabador de la asignatura. En cada toma y gesto, los primeros 70 fotogramas van a entrenamiento y los 30 últimos a test. Todas las personas están en los dos lados.

- **LOPO** (*leave-one-participant-out*): se entrena con cuatro personas y se evalúa con la quinta, rotando las cinco. Mide lo que importa en la demo: reconocer a alguien nuevo. **Es el protocolo de referencia de toda la memoria.**

La validación para la parada temprana sale siempre de la parte de entrenamiento (los últimos fotogramas de cada toma y gesto), nunca del test. El notebook de la asignatura usaba el propio test como validación, de modo que el test acababa eligiendo el modelo; lo corregimos en nuestro pipeline.

5.2 Métricas

Accuracy, F1 por clase y F1 macro, accuracy equilibrada, matriz de confusión y accuracy por participante. Cada configuración se repite con tres semillas (dos en los experimentos de aumentación) y se da la media y la desviación entre participantes. Para medir la dependencia de la mano, cada imagen de test se refleja además horizontalmente (la misma pose hecha con la otra mano) y se vuelve a evaluar.

6 Resultados

6.1 La normalización es la decisión que más pesa

Modelo	Ninguna	Muñeca	Muñeca + escala	Muñeca + escala + giro
CNN del enunciado	90,9 $\pm$ 5,2	94,8 $\pm$ 1,8	94,8 $\pm$ 1,5	95,8 $\pm$ 2,2
MLP	90,3 $\pm$ 5,6	94,4 $\pm$ 2,9	95,2 $\pm$ 1,6	96,2 $\pm$ 2,0
SVM	87,1 $\pm$ 7,7	90,2 $\pm$ 4,9	94,0 $\pm$ 2,2	94,7 $\pm$ 2,9
Random forest	82,6 $\pm$ 9,2	84,0 $\pm$ 7,4	90,6 $\pm$ 3,9	93,4 $\pm$ 4,5
k vecinos	68,0 $\pm$ 12,1	76,7 $\pm$ 10,5	88,0 $\pm$ 5,0	91,0 $\pm$ 5,0
Regresión logística	90,4 $\pm$ 4,0	91,1 $\pm$ 3,0	92,2 $\pm$ 3,0	88,5 $\pm$ 5,0

Tabla 4: Accuracy en LOPO (%), media $\pm$ desviación entre participantes, sin aumentación.

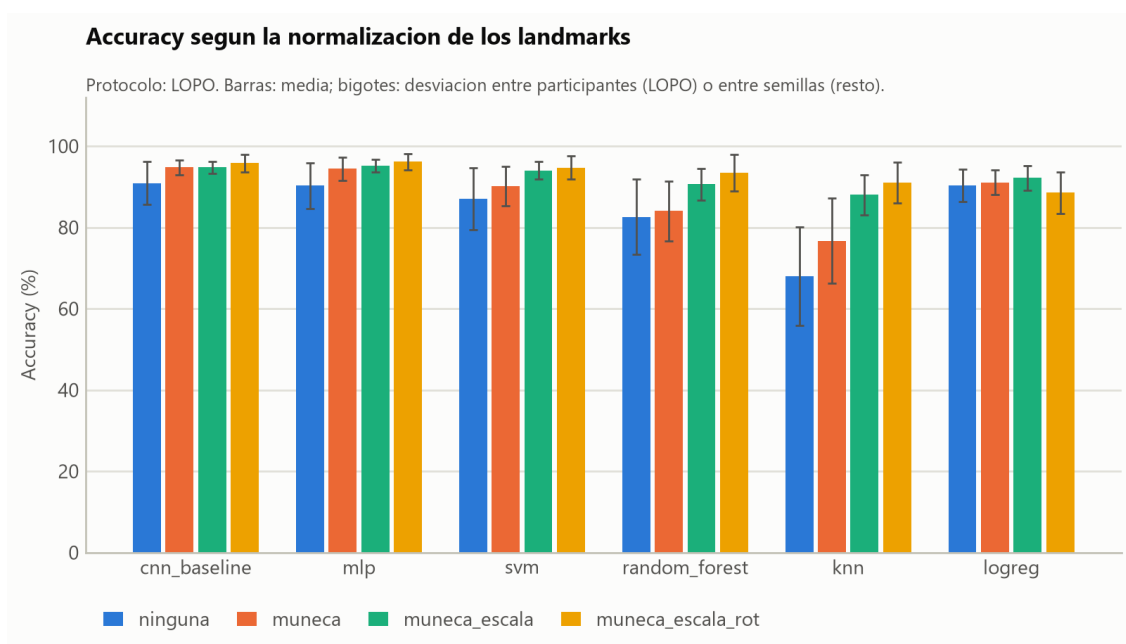


Figura 2: Accuracy en LOPO de cada modelo con cada normalización.

Todos los modelos salvo la regresión logística mejoran al pasar de las coordenadas sin tocar a la normalización completa, y la desviación entre personas se reduce a la mitad o menos: el modelo deja de depender de cómo graba cada uno. Los que más ganan son los que solo miden distancias o umbrales sobre las coordenadas: k vecinos sube 23 puntos y el random forest, 11. La regresión logística, el único modelo lineal, es la excepción: rinde mejor sin el giro.

6.2 Evaluar con personas nuevas

Modelo	Normalización	Temporal	LOPO	Temporal – LOPO
CNN del enunciado	muñeca + escala + giro	94,3	95,8	−1,5
MLP	muñeca + escala + giro	95,5	96,2	−0,7
SVM	muñeca + escala + giro	92,9	94,7	−1,8
Random forest	muñeca + escala + giro	92,6	93,4	−0,9
k vecinos	muñeca + escala + giro	90,4	91,0	−0,6
Regresión logística	muñeca + escala	92,8	92,2	+0,7

Tabla 5: Misma configuración evaluada con el protocolo temporal y con LOPO (%).

Con los datos normalizados, el protocolo temporal y LOPO dan cifras parecidas (diferencia de 1,8 puntos como mucho). El test temporal son los últimos 30 segundos de cada gesto, con posturas del final de la toma, así que tampoco es un test fácil. Aun así, solo LOPO responde a la pregunta de la demo, y es el que usamos para elegir.

6.3 Experimentos de invarianza y aumentación

Experimento	MLP		CNN del enunciado	
	LOPO	Otra mano	LOPO	Otra mano
Principal (sin aumentar)	96,2	96,4	95,8	96,0
Espejo por lateralidad	95,9	95,9	95,9	95,9
Aumentación (3 copias)	96,1	96,2	95,8	95,9
Aumentación con volteo	96,5	96,6	96,2	96,2

Tabla 6: Accuracy (%) con la normalización completa. «Otra mano» es el test reflejado horizontalmente.

Ningún modelo pierde acierto con la mano contraria, ni siquiera sin aumentación: con la normalización, una mano izquierda y una derecha haciendo el mismo gesto ya se parecen lo suficiente. La aumentación con volteo es la que da el mejor resultado y la que elegimos. El espejo por lateralidad no aporta, como cabía esperar de una etiqueta poco fiable.

6.4 Errores por gesto

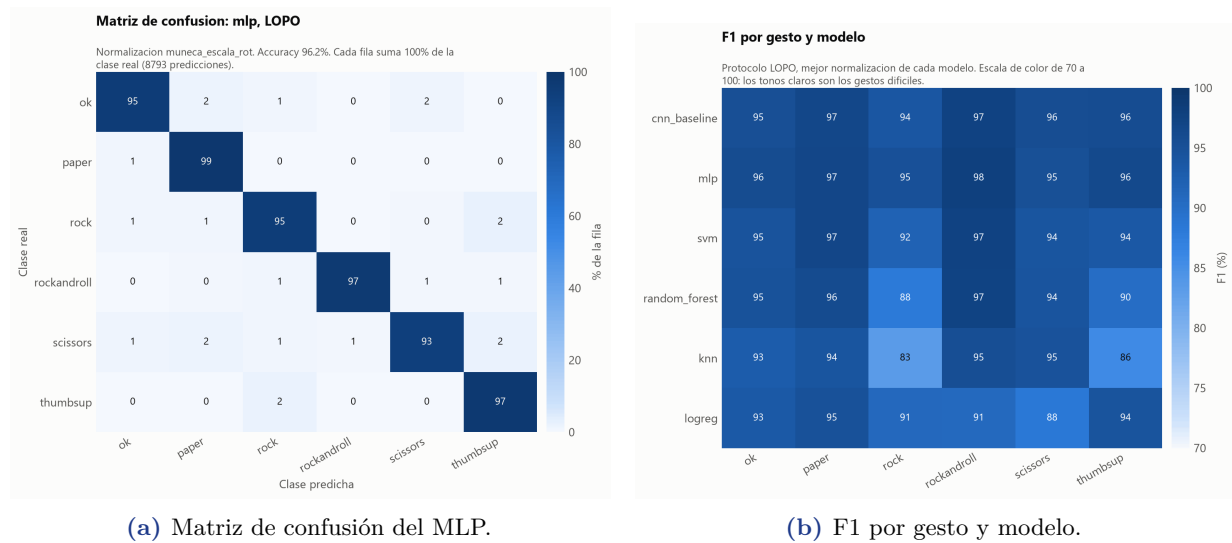


Figura 3: Errores por gesto en LOPO con la normalización completa.

El MLP no baja del 93 % de acierto en ningún gesto. Sus confusiones más frecuentes son **rock** tomado por **thumbsup** (2,4 % de los **rock**), **scissors** por **thumbsup** (2,0 %), **thumbsup** por **rock** (2,0 %) y **ok** por **paper** (1,9 %): los pares que la guía de grabación ya señalaba. Los modelos más simples sufren en los mismos gestos: k vecinos se queda en un F1 de 83 en **rock** y de 86 en **thumbsup**.

6.5 Acierto por persona

Persona no vista	p1	p2	p3	p4	p5
MLP, normalización completa	94,2	97,6	97,5	97,8	93,8
CNN del enunciado, sin normalizar	87,8	94,9	93,3	95,4	83,3

Tabla 7: Accuracy (%) con cada persona fuera del entrenamiento.

p1 y p5 son las personas más difíciles: son las que grababan la mano más pequeña, es decir, más lejos de la cámara (tabla 2). Son también las que más ganan con la normalización: p5 pasa del 83,3 % al 93,8 %.

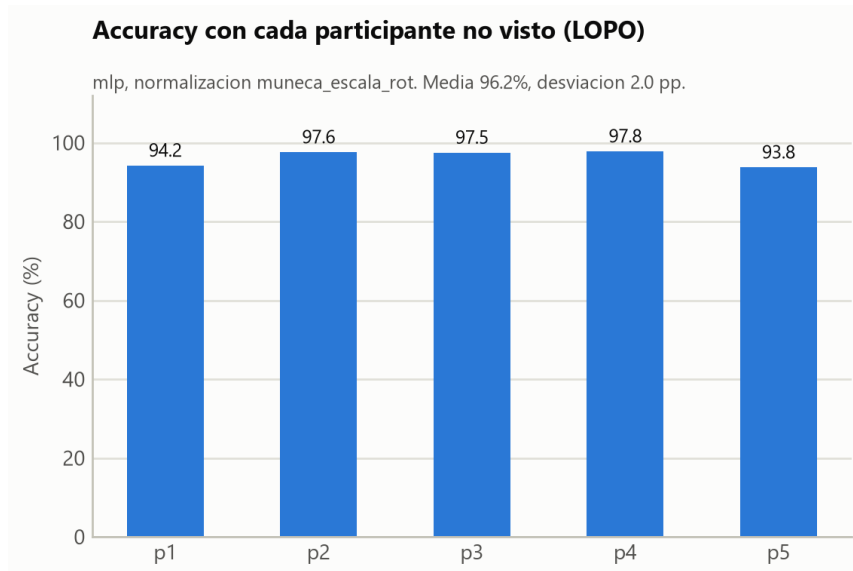


Figura 4: Accuracy del MLP con cada persona no vista.

6.6 Coste computacional

Modelo	Parámetros	Tamaño (KB)	Latencia (ms)
MLP	5030	86	0,84
CNN del enunciado	23 126	303	1,23
SVM	—	200	0,16
Regresión logística	—	3	0,14
Random forest	—	7546	6,26
k vecinos	—	304	10,32
Extractor de MediaPipe	—	7636	16,07

Tabla 8: Coste por muestra con lote de 1 y un solo hilo de CPU, como en la demo (mediana de 300 a 1000 repeticiones).

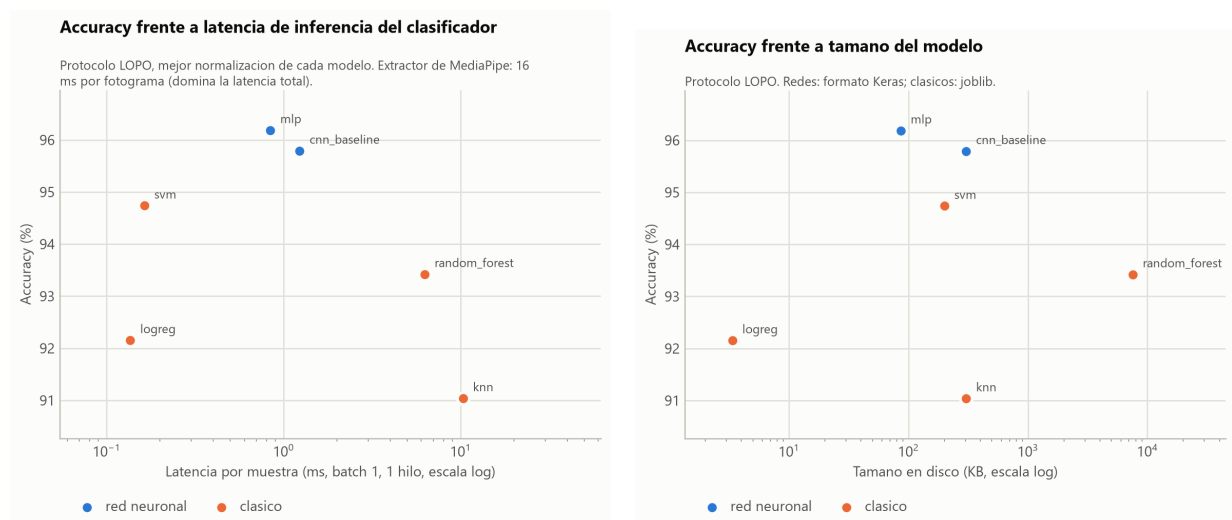


Figura 5: Acierto frente a latencia y frente a tamaño en disco.

La latencia la marca MediaPipe (16,1 ms), no el clasificador: con el MLP el techo teórico es de unos 59 fotogramas por segundo. Entre los modelos, el MLP es el único que está a la vez en lo más alto en acierto y por debajo de 1 ms y de 100 KB. El random forest ocupa 7,5 MB y k vecinos guarda todo el entrenamiento y tarda más de 10 ms.

7 Modelo elegido y despliegue

Elegimos el **MLP con la normalización completa y aumentación con volteo**: 96,5 % en LOPO y 96,6 % con la otra mano. Frente a la CNN del enunciado acierta un poco más con cuatro veces menos parámetros, menos de un tercio de tamaño y menos latencia. La convolución no aporta porque la entrada ya no es una imagen sino 42 coordenadas normalizadas.

- **Demo en tiempo real** (demo/src/demo-gestures-PIDS.py): abre la webcam, dibuja los puntos y muestra el gesto y la orden del tanque.
- **En el navegador**: los pesos se exportan a JSON y el portal web ejecuta MediaPipe para web y el MLP en TypeScript, sin servidor. Con 24 muestras reales del dataset, las probabilidades coinciden con las de Python a 5 decimales.
- **Integración con la plataforma**: a la plataforma de datos solo llega el nombre del gesto y su confianza, nunca la imagen ni los puntos de la mano. Encaja con el escenario E3 de privacidad de las fases 2 y 3.

8 Análisis sin etiquetas

Como análisis complementario comprobamos si los gestos se separan solos, sin dar ninguna etiqueta al algoritmo. Agrupamos las 2931 manos con k -means y con el método de Ward y las proyectamos en dos dimensiones con t-SNE. El acuerdo entre grupos y etiquetas se mide con el índice de Rand ajustado (ARI: 0 equivale al azar y 1 a grupos idénticos). La tercera representación añade a la normalización completa el espejo por lateralidad.

Representación	ARI con el gesto	Manos en el grupo de su gesto	ARI con la persona
Sin normalizar	0,05	27,8 %	0,05
Normalizada	0,22	37,6 %	0,01
Normalizada + espejo	0,36	49,9 %	0,03

Tabla 9: k -means con 6 grupos (uno por gesto) y con 5 (uno por persona).

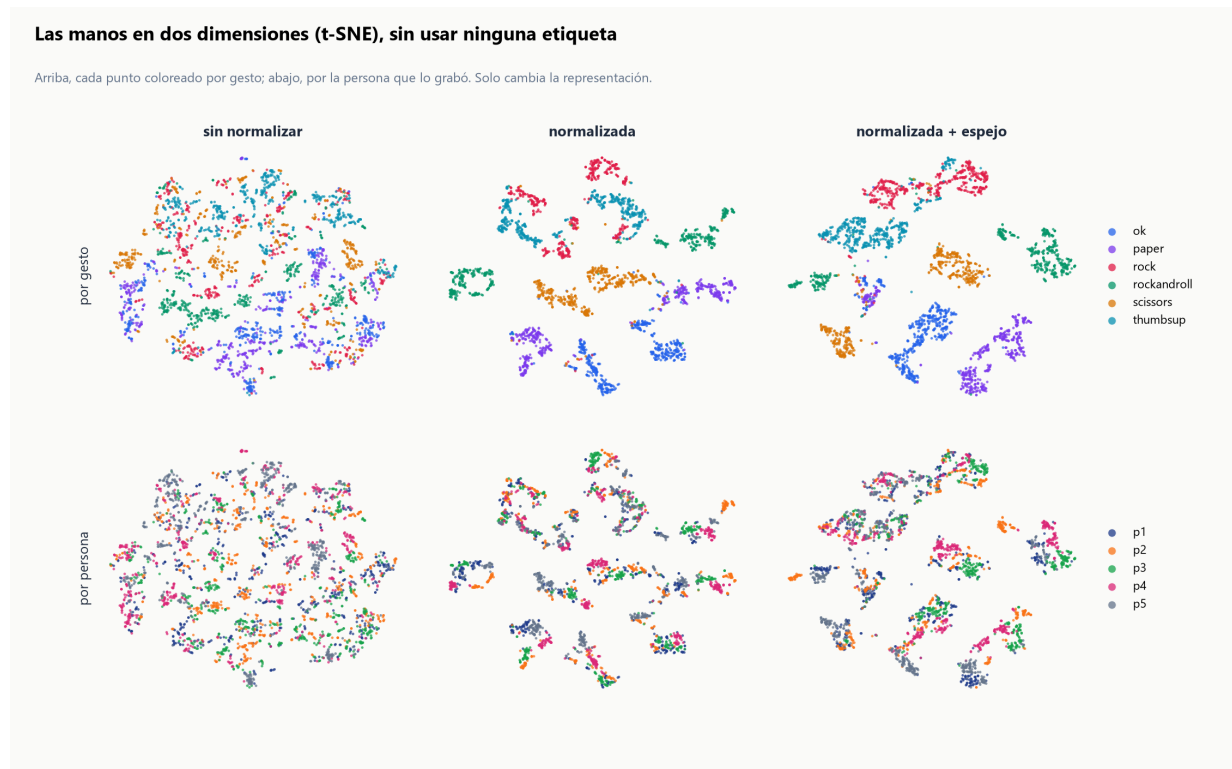


Figura 6: Proyección t-SNE de las manos con cada representación, coloreada por gesto (arriba) y por persona (abajo).

- **La normalización hace aparecer los gestos:** sin normalizar todo está mezclado; normalizado aparecen islas de un solo gesto, y el acuerdo con los gestos se multiplica por siete.
- **Las personas no se agrupan** en ninguna representación (ARI entre 0,01 y 0,05): lo que distingue a las manos es el gesto, no quién lo hace.
- **Cada gesto tiene variantes:** con 6 grupos solo la mitad de las manos cae en el de su gesto (por ejemplo, ok y paper acaban juntos), pero con más grupos la cifra sube deprisa: 83 % con 12 y 91 % con 20, lo mismo que k vecinos supervisado (figura 7).

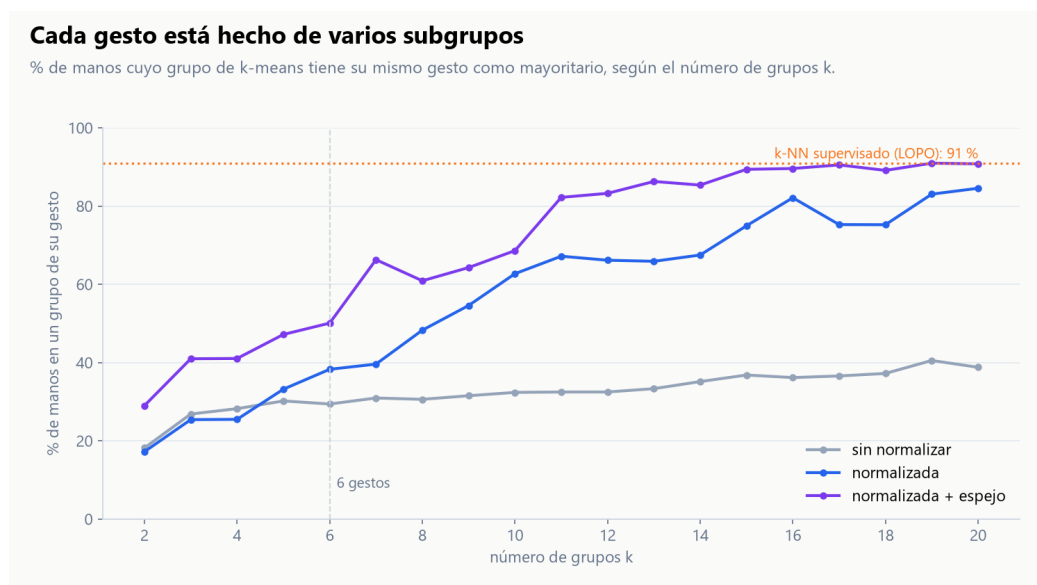


Figura 7: Porcentaje de manos cuyo grupo de k -means tiene su gesto como mayoritario, según el número de grupos.

La conclusión es que los gestos tienen estructura propia, pero hace falta un clasificador supervisado para unir las variantes de cada uno.

9 Limitaciones

- **Cinco personas y pocas condiciones:** las tomas se grabaron en dos días y en entornos parecidos. Con más personas, luces y cámaras la estimación de LOPO sería más fiable; su desviación entre personas (± 2 puntos) indica cuánto puede variar con alguien nuevo.
- **Sin clase de «ningún gesto»** en la evaluación: el clasificador siempre elige uno de los seis.
- **Solo coordenadas 2D:** no usamos la profundidad estimada por MediaPipe.
- **Sin TensorFlow Lite:** no se pudo generar porque la política de seguridad del portátil bloquea TensorFlow. Para el navegador lo resolvimos exportando el MLP a JSON.

10 Conclusiones

1. **La representación importa más que el modelo:** con los mismos datos, normalizar sube todos los modelos y reduce a la mitad la diferencia entre personas.
2. **Medir con personas nuevas cambia las conclusiones:** LOPO es la única cifra que dice qué pasará en una demostración, y con ella la diferencia entre la CNN y el MLP es pequeña.
3. **Lo sencillo gana:** un MLP de 86 KB acierta tanto o más que la CNN, corre en menos de 1 ms y se ejecuta en el navegador, lo que permitió integrar la fase 1 en el portal sin enviar ninguna imagen.

A Reproducibilidad

Todo el código está en la carpeta `parte1_gestos/` del repositorio:

Ruta	Contenido
<code>kit_grabacion/</code>	Grabador del dataset y su guía
<code>entrenamiento/</code>	Preprocesado, rejilla de experimentos, evaluación y exportación (paquete <code>hgr</code> , con tests)
<code>entrenamiento/clustering.py</code>	Análisis sin etiquetas de la sección 8
<code>datos_generados/</code>	Puntos de la mano de las 3000 imágenes
<code>resultados/</code>	Informe, tablas y figuras de cada experimento, comparativa y clustering
<code>modelos/</code>	MLP y CNN exportados
<code>demo/</code>	Demo en tiempo real y notebooks

Con el dataset descomprimido y `PIDS_DATOS` apuntando a él, se reproduce con:

```
python parte1_gestos\descargar_modelos.py
python parte1_gestos\entrenamiento\preprocesar.py
python parte1_gestos\entrenamiento\entrenar_todo.py
python parte1_gestos\entrenamiento\clustering.py
```

Versiones: Python 3.12, MediaPipe 0.10.35, Keras 3.15.1 (sobre PyTorch), scikit-learn 1.9.0.